# Precision-AMR: Spatially Adaptive Floating-Point Precision for PDE-Based Reverse Time Migration

*A new axis of numerical adaptivity for adjoint wave-equation solvers, developed on dedicated NVIDIA hardware and validated across heterogeneous NVIDIA and AMD GPU architectures on the Santos Dumont supercomputer (LNCC)*


**Principal Investigator:** Prof. Sandro Rigo
**Title/Role:** Associate Professor, Institute of Computing
**University:** Universidade Estadual de Campinas (UNICAMP)

**Collaborator 1:** Thiago Maltempi — Graduate Researcher (PhD Candidate), Institute of Computing, Universidade Estadual de Campinas
**Collaborator 2:** Prof. Hervé Yviquel — Assistant Professor, Institute of Computing, Universidade Estadual de Campinas


## 1 Problem Statement

GPU hardware is increasingly shifting toward low precision in favor of AI workloads (FP8/BF16), exposing new memory hierarchies, tensor cores, and mixed-precision math libraries. At the same time, scientific simulations based on partial differential equations (PDEs) continue to require FP64/FP32 precision to remain trustworthy. Reverse Time Migration (RTM), a seismic imaging method that solves the wave equation by finite differences over thousands of time steps, is a canonical example of this tension: geophysicists use the migrated image to guide expensive drilling decisions, so numerical fidelity is essential. RTM is also memory-intensive, requiring hundreds of gigabytes of wavefield state and relying on checkpointing to trade memory against recomputation in its forward/backward adjoint structure.

Our prior work tackled this structure's main bottleneck, namely host–GPU checkpoint transfer over PCIe, by combining the concept of prefetching with data compression. This idea was materialized as the GPUZIP library [3, 4], which compresses checkpoints on the GPU and prefetches them proactively, reducing host–GPU transfer volume by up to 80$\times$ and achieving end-to-end speedups of up to 5.1$\times$ on V100, while preserving image quality (SSIM $\approx$ 1, near-zero relative error). Separately, reduced-precision arithmetic has already been shown to be viable for seismic modeling, RTM, and FWI: Fabien-Ouellet [1] found that a stable half-precision (FP16) implementation of the elastic wave equation is achievable through appropriate scaling, that the resulting seismogram error is a negligible fraction of total seismic energy and largely incoherent with seismic phases, and that this noise does not adversely affect FWI or RTM image quality. However, that reduction is applied uniformly across the entire computational domain. Gao and Harms [2] identified a specific hazard of that approach: naive half-precision time stepping accumulates round-off error through a disguised recursive summation in the update rule, degrading solution quality, with compensated (Kahan) summation as an effective remedy.

At the same time, adaptive mesh refinement (AMR) has been explored for seismic wave simulation and, more recently, explicitly for FWI and RTM workflows, to avoid the waste inherent in a uniform grid: dispersion and dissipation errors require grid spacing proportional to the shortest wavelength, so a uniform mesh is forced to use the finest spacing everywhere, even where the local velocity field would tolerate a much coarser one. AMR addresses this by refining $\Delta x$ locally. No comparable, physically motivated scheme exists for floating-point precision, however: existing mixed-precision work in seismic imaging is either global and uniform (as above), or adaptive only at the level of an individual linear-algebra operation (mixed-precision GEMM, iterative refinement, Ozaki-scheme emulation), not tied to a spatial decomposition of the wavefield itself. None of this prior work varies floating-point precision according to physically motivated spatial criteria. This is therefore a third axis of adaptivity that remains largely unexplored.

This project proposes to close that gap with Precision-AMR (P-AMR): a spatially block-structured scheme, an adaptive-precision analogue of AMR, that keeps mesh resolution fixed and instead varies the floating-point format assigned to each block (tile) of the domain, based on physically motivated error-tolerance criteria (proximity to absorbing boundaries, wavefield energy density, adjoint-sensitivity magnitude, and a time-dependent band trailing the propagating wavefront) that predict whether the round-off error introduced in that block can reach the imaging condition or the FWI gradient. This is a structurally different problem from mesh-based AMR. In AMR, the characteristic artifact is a localized reflection at the coarse-fine grid interface, because refinement changes the discretization itself. In P-AMR, no discretization boundary is introduced; instead, a spatially varying level of round-off noise is injected into a hyperbolic PDE, and that noise propagates together with the wave. Tagging criteria therefore cannot rely solely on local smoothness, as AMR refinement indicators do; they must instead approximate where an error, once injected, will decay, be absorbed, or otherwise fail to reach the receivers or the sensitivity kernel before the simulation ends, and control round-off accumulation over the thousands of time steps typical of RTM/FWI.

Because RTM/FWI already require a block/tile decomposition of the domain for checkpointing, P-AMR reuses that same decomposition as its tagging granularity, unifying the precision budget and the lossy-compression budget of differential checkpointing into a single, jointly tuned per-tile accuracy target. Mixed-precision GEMM, cuSOLVER iterative refinement, and Ozaki-scheme FP64 emulation remain essential as supporting techniques, but are now applied selectively, only where P-AMR's criteria indicate they are needed — the natural way to exploit GPUs geared primarily toward AI, such as the RTX PRO 6000 Blackwell, which offer negligible native FP64 throughput.

Because the tagging criteria are motivated by the physics of wave propagation and adjoint sensitivity, rather than by any particular vendor's arithmetic units, we treat their portability across architectures as an explicit question, not an assumption. Our research group has access to the Santos Dumont supercomputer (LNCC/SINAPAD), which includes AMD Instinct MI300A accelerators alongside several NVIDIA generations (Volta V100, Hopper H100, Grace-Hopper GH200). This lets us test that question directly, rather than merely assert it, and it is what gives Precision-AMR its potential contribution to the field: a numerical adaptivity axis that is new and portable across vendors, not merely a one-off optimization for a single GPU architecture.

## 2 Expected Results

The project's principal contribution is the Precision-AMR (P-AMR) method itself, which explores a new axis of numerical adaptivity that varies floating-point precision spatially, according to physically motivated criteria (proximity to the PML, wavefield energy density, adjoint-sensitivity magnitude, and a wavefront-trailing band), proposed and validated specifically for adjoint PDE solvers such as RTM/FWI. This contribution includes the formulation of the four tagging criteria, the characterization of the "precision-interface" artifact, analogous to the coarse-fine interface reflections of classical AMR, and the demonstration that the precision and lossy-compression budgets can be unified into a single per-tile error target.

As secondary contributions, the project will produce: (i) a multi-architecture evaluation of P-AMR-guided mixed-precision usage, comparing NVIDIA GPUs (V100, H100, GH200) and the AMD Instinct MI300A APU, characterizing how much of the achieved speedup and accuracy trade-off is vendor-independent; and (ii) the implementation of P-AMR as an **independent library**, with an interface that allows its integration into other PDE-based simulation applications, beyond the specific scope of RTM/FWI. This library may be adopted standalone or combined with GPUZIP, merging P-AMR's spatial precision zoning with GPUZIP's compressed checkpointing into a single workflow, without requiring the host application to adopt the entire GPUZIP stack just to benefit from P-AMR. It will be made available as open source, under the MIT license, on GitHub.

The central scientific result will be submitted to a high-impact high-performance-computing event or journal, for example, Supercomputing (SC), IPDPS, Euro-Par, or SBAC-PAD among the events, or IJHPCA or a Transactions-type journal from IEEE or ACM among the journals, accompanied by a public reproducibility package (datasets, Nsight profiles, quality metrics), made available via UNICAMP's REDU.

## 3 Computational Platforms

**HPC SDK** (nvc++/nvfortran, CUDA-X math libraries, Nsight Systems/Compute). Experience: team uses CUDA and Nsight Systems routinely. Adoption: migrate the Awave-3D build from clang to nvc++ and validate numerical parity against existing baselines. Nsight Compute provides kernel-level roofline analysis, now applied per spatial tile, to identify which tiles are memory-bandwidth-bound versus compute-bound and to objectively scope where a lower-precision tile actually pays off.

**RTX PRO 6000 Blackwell** (dedicated hardware, requested). This GPU's 5th-generation tensor cores with FP8/FP4 support and negligible native FP64 throughput make it the central platform for P-AMR: because full-domain FP64 emulation would be expensive everywhere, the scientific question this project answers is precisely which tiles need it and which do not. Its 96 GB GDDR7 ECC memory also supports local development; as dedicated, queue-free hardware it enables fast iterative debugging that a shared batch system is not well suited for.

**Santos Dumont** (LNCC/SINAPAD, Tier-0 national supercomputer, Petrópolis-RJ). Provides the large-scale validation environment. Its BullSequana X1000 partition includes 94 nodes with 4$\times$ NVIDIA Volta V100 GPUs each — matching the hardware used for the original GPUZIP baselines — plus a dedicated AI node with 8$\times$ V100 16GB over NVLink. Its newer BullSequana XH3000 partition adds 62 nodes with 4$\times$ NVIDIA Hopper H100 SXM 80GB over NVLink, 36 NVIDIA Grace-Hopper GH200 superchip nodes, and 18 nodes with 2$\times$ AMD Instinct MI300A APUs (CDNA3 GPU cores, 128 GB HBM3 each). This heterogeneity lets us generate FP64 ground truth and run the full P-AMR criteria sweep at scale on NVIDIA hardware, and separately test whether the tagging methodology and block-tiling approach generalize to a non-NVIDIA architecture.

**nvCOMP / Bitcomp** (GPU-side lossy compressor). Experience: integrated and validated in GPUZIP v2.0, where Bitcomp delivered the best accuracy-compression balance on V100. Adoption here: the storage mechanism for differential checkpointing, now indexed by the same tile identifiers used for precision tagging, so each tile carries a single, jointly-tuned precision-and-compression error budget.

**cuBLAS (mixed-precision extensions) and cuSOLVER (iterative refinement).** Both ship inside the HPC SDK. Experience: team has used BLAS/LAPACK routines in prior HPC work. Adoption: applied only within tiles that P-AMR flags as containing dense subkernels needing accuracy beyond native low precision (FP32 compute with FP64 accumulation, and cuSOLVER iterative refinement).

**Ozaki-scheme FP64 emulation** via low-precision tensor cores. Adoption: reserved for the subset of tiles that P-AMR's sensitivity criteria mark as accuracy-critical (near the wavefront, high adjoint-sensitivity magnitude), rather than applied uniformly — directly testing whether selective emulation is cheaper than emulating everywhere while preserving image fidelity. Recent work has demonstrated up to 2.3× speedup over native FP64 DGEMM on the RTX PRO 6000 Blackwell [5].

**cuFFT.** Experience: used in prior spectral operator work. Adoption: benchmarked as an alternative to FDM stencils for wavefield updates within selected, precision-tagged tiles, characterising the memory-bandwidth versus compute trade-off relative to the standard finite-difference path.

**AMD Instinct MI300A** (CDNA3, Santos Dumont, exploratory). As an additional validation activity, we assess how much of the P-AMR framework — the block/tile decomposition, the four tagging criteria, and the joint precision-compression error budget — transfers to AMD's ROCm/HIP ecosystem (rocBLAS, hipBLAS, and AMD's own reduced-precision matrix paths), without assuming feature parity with the CUDA-specific Ozaki-scheme emulation or nvCOMP/Bitcomp compression path. The goal is to determine whether Precision-AMR is a portable methodology or an NVIDIA-specific optimization.

## 4 Dataset and Model

- Size: ≈X TB total (to be confirmed from REDU). Checkpoint pools reached ≈330 GB host RAM on V100; a single Marmousi3D checkpoint is ≈500 MB.
- Data type: seismic velocity models and traces (numerical grids).
- Source: Salt (SEG/EAGE) and Marmousi3D are open datasets; the Large dataset is in-house.

- Confidentiality: non-confidential, no personal data, no NDA.
- AI model parameters: not applicable (numerical PDE solver).
- Storage location: Universidade Estadual de Campinas (UNICAMP), Campinas, Brazil; staged for upload to Santos Dumont's parallel file systems for large-scale runs.

# 5 Methods

## 5.1 Phase 1 — Infrastructure and Baseline (M1–M2)

Port Awave-3D and GPUZIP to the HPC SDK (nvc++), re-integrate nvCOMP/Bitcomp, and reproduce V100 baselines on Santos Dumont's V100 partition, matching the original GPUZIP hardware.

- Establish a block/tile decomposition of the domain, extending the existing checkpoint-block structure so that it can also carry a per-tile precision tag.
- Profile every kernel with Nsight Systems and Nsight Compute, on both the RTX PRO 6000 and Santos Dumont, confirming that FDM stencils are memory-bandwidth-bound at the tile granularity before any precision changes are attempted.

## 5.2 Phase 2 — Precision-AMR: Criteria, Tagging, and Validation (M2–M6)

This is the core scientific phase, run at scale on Santos Dumont's H100 partition. We design, implement, and validate four candidate tagging criteria for spatial precision zoning:

- **Boundary criterion:** tiles inside or adjacent to the PML/absorbing layer are tagged for reduced precision by default, since their contribution is already being damped.
- **Energy criterion:** tiles are tagged by a threshold on local wavefield energy density, so that regions of low amplitude (before wavefront arrival, or after geometrical spreading has attenuated the signal) tolerate lower precision.
- **Sensitivity criterion (FWI):** tiles are tagged using the magnitude of the local adjoint-sensitivity kernel, reusing the gradient computation itself as the tagging signal, in direct analogy to a Berger–Oliger truncation-error estimator in classical AMR.
- **Wavefront-trailing criterion:** a time-dependent band immediately behind the propagating wavefront is tagged for reduced precision, while the leading edge — where phase accuracy governs the imaging condition — retains full precision.

For each criterion, we implement compensated (Kahan) summation in the low-precision tiles' time-stepping update to control round-off accumulation over the full simulation, following the mechanism identified by Gao and Harms [2]. We explicitly test for a "precision-interface" artifact — analogous to coarse-fine grid-interface reflections in mesh-based

AMR — via residual analysis at tile boundaries between differing precisions. Finally, each tile is assigned jointly a floating-point format and a checkpoint compression policy (full snapshot versus compressed delta), with combined accuracy measured by PSNR, SSIM, and mean relative error against the FP64 baseline, on both the raw wavefield and the final migrated image. This phase subsumes differential checkpointing, which becomes the storage mechanism for P-AMR-tagged tiles rather than a separate deliverable.

### 5.3 Phase 3 — Precision-AMR without Native FP64 (M7–M10)

The tagging criteria validated in Phase 2 are applied on the RTX PRO 6000, hardware with negligible native FP64 throughput. Per tile, the pipeline selects between native low-precision tensor-core execution, cuBLAS mixed-precision GEMM (FP32 compute with FP64 accumulation) or cuSOLVER iterative refinement for dense sub-kernels, and Ozaki-scheme FP64 emulation for tiles flagged as accuracy-critical.

- This directly operationalises the motivation for exploiting AI-first GPUs that lack native FP64: expensive emulation is concentrated only where P-AMR's criteria say it is needed.
- We quantify the speedup/accuracy trade-off relative to a uniform-Ozaki baseline (emulating FP64 everywhere) to determine how much selective tiling actually saves.
- All errors are bounded jointly (compression $\times$ precision interaction) against the FP64 baseline using PSNR, SSIM, and mean relative error, across the same three datasets and three checkpointing algorithms (Revolve, zCut, Uniform) used throughout the project.
- Exploratory cross-architecture activity: the same tagging and tiling logic is ported to Santos Dumont's AMD Instinct MI300A nodes via ROCm/HIP, to test whether the criteria and error-budget framework hold on a CDNA3 architecture without native access to Ozaki-scheme emulation or nvCOMP/Bitcomp.

### 5.4 Phase 4 — Spectral Operators, Integration, and Release (M10–M12)

- Benchmark cuFFT-based pseudo-spectral derivative operators as an alternative update rule for selected, precision-tagged tiles, as a secondary technique complementing the core P-AMR contribution.
- Integrate the Precision-AMR module, differential checkpointing, and mixed-precision paths into a unified GPUZIP v3.0 release.
- Multi-node scaling evaluation on Santos Dumont ($\leq$8 GPUs).

- Final analysis, write-up, and submission, with Precision-AMR as the central scientific claim and differential checkpointing, mixed-precision linear algebra, and spectral operators presented as supporting techniques.

## 6 Evaluation

All experiments span three datasets (Large, Marmousi3D, Salt), three checkpointing algorithms (Revolve, zCut, Uniform), and the four P-AMR tagging criteria (individually and in combination). Kernel-level profiling uses Nsight Compute (roofline, memory-traffic breakdowns); timeline profiling uses Nsight Systems. Implementation is in C/C++ with CUDA under the HPC SDK, with OpenMP Cluster for multi-node scaling. Accuracy is evaluated on both the wavefield and the final RTM image, since a tiling that preserves wavefield PSNR/SSIM may still, in principle, shift the migrated image if error is spatially correlated with reflectors. For the cross-architecture activity, the same accuracy metrics are computed on Santos Dumont's AMD MI300A partition and compared against the NVIDIA baselines, to characterise how much of the achieved speedup and accuracy trade-off is architecture-independent.

## 7 Project Support Details

We request one RTX PRO 6000 Blackwell GPU as dedicated local development hardware. Its abundant FP8/FP4 tensor-core throughput and negligible native FP64 make it the scientifically relevant target for the selective-emulation study central to Phase 3: it forces the question of whether Precision-AMR's selective use of Ozaki-scheme emulation can match full-domain emulation at a fraction of the cost. As dedicated, queue-free hardware, it also supports the iterative day-to-day development that a shared batch system is not well suited for. Large-scale validation — the P-AMR criteria sweep across datasets and checkpointing algorithms, FP64 baseline generation, multi-GPU scaling, and the AMD MI300A portability test — runs separately on the Santos Dumont supercomputer (LNCC/SINAPAD), accessed through LNCC's own allocation program and outside the scope of this request.

### Resource Request

| Resource | Request |
|---|---|
| Physical hardware | 1$\times$ RTX PRO 6000 Blackwell — dedicated local development hardware, used throughout the project |

Compute time on Santos Dumont is requested separately through LNCC's own project allocation program (SINAPAD) and is not part of this hardware request.

## References


[1] Gabriel Fabien-Ouellet. Seismic modeling and inversion using half-precision floating-point numbers. *Geophysics*, 85(3):F65–F76, 2020.

[2] Longfei Gao and Kevin Harms. Half precision wave simulation. arXiv:2310.00236, 2023. Revised 2024.

[3] LSC-Unicamp. GPUZIP source code. `https://github.com/LSC-Unicamp/GPUZIP`, 2025. Accessed 2026.

[4] Thiago Maltempi, Sandro Rigo, Marcio Pereira, Hervé Yviquel, Gustavo Leite, Otavio Lee, Jesse Costa, and Guido Araujo. Checkpointing fine-tuning for accelerating seismic applications in GPUs. *International Journal of High Performance Computing Applications*, 39(6):784–802, 2025.

[5] Takumi Uchino et al. Guaranteed DGEMM accuracy while using reduced precision tensor cores through extensions of the Ozaki scheme. arXiv:2511.13778, 2025. Presented at SCA/HPCAsia 2026, Osaka, Japan.